Mathematical Modeling

Chapter 1: Modeling Optimization Problems

Personal study notes

Contents

1 What mathematical programming means **1**

2 A concrete model: five products and three resources **1**
2.1 A hand solution with a reason it is optimal 2

3 The general constrained problem **3**

4 From a real question to a mathematical model **3**
4.1 Units and assumptions 3

5 How optimization problems are classified **4**

6 Optimization roadmap **4**

7 MATLAB refresher scope **5**

8 Questions for revision **5**

9 Sources and editorial remarks **5**

1 What mathematical programming means

Here, *mathematical programming* means mathematical optimization: selecting the best feasible decision according to a specified criterion. The word "programming" refers to planning; an optimization problem is a mathematical object even before any computer program is written.

A model has three central ingredients:

1. **Decision variables**: quantities that may be chosen.
2. **Constraints**: conditions that an acceptable decision must satisfy.
3. **Objective function**: a numerical criterion to minimize or maximize.

Parameters, such as available resources or prices, are fixed data for one model instance. A parameter can change between instances without becoming a decision variable.

2 A concrete model: five products and three resources

Suppose a factory chooses weekly amounts $x_1, \ldots, x_5$ of five products. Treat these amounts as nonnegative real quantities: this is a continuous model whose divisibility assumption must be justified for the actual products. Machine use and profit per unit are fixed and additive:

Product	1	2	3	4	5
Profit (PLN/unit)	550	600	350	400	200
Grinding (h/unit)	12	20	0	25	15
Drilling (h/unit)	10	8	16	0	0
Technician work (h/unit)	20	20	20	20	20

Each machine and technician works $6 \cdot 8 = 48$ hours per week. Two drilling machines supply 96 machine-hours, and eight technicians supply 384 person-hours. Let G denote available grinding hours; three machines give $G = 144$, whereas four give $G = 192$. Assume capacity can be pooled, with no additional scheduling, setup, or demand restrictions. Each coefficient times x_j is a required number of hours, so the model is

$$\begin{aligned} \max \quad & 550x_1 + 600x_2 + 350x_3 + 400x_4 + 200x_5 \\ \text{subject to} \quad & 12x_1 + 20x_2 + 25x_4 + 15x_5 \le G, \\ & 10x_1 + 8x_2 + 16x_3 \le 96, \\ & 20(x_1 + x_2 + x_3 + x_4 + x_5) \le 384, \qquad x \ge 0. \end{aligned} \tag{1}$$

Resource limits give upper bounds, while $x_j \ge 0$ prevents negative production. The objective counts total profit, using the stated unit profits. Choosing a single product by its profit per unit alone ignores the different resources it uses. For a general indexed model, let row i represent a resource and column j a product: a_{ij} has units of resource i per unit of product j. Then row i is $\sum_j a_{ij}x_j \le b_i$, or $Ax \le b$. Rows combine consumption of one resource across all products; columns describe all resources used by one product.

2.1 A hand solution with a reason it is optimal

For $G = 144$ or 192, consider

$$x_1 = \frac{48}{5}, \quad x_4 = \frac{G - 576/5}{25}, \quad x_2 = x_3 = x_5 = 0.$$

Grinding and drilling are used exactly; technician work is below 384 hours. To compare this plan with *every* feasible plan, multiply the grinding inequality by 16 and the drilling inequality by 179/5. Their combined left side has coefficients

$$(550,\ 3032/5,\ 2864/5,\ 400,\ 240),$$

each at least the corresponding profit coefficient. Since $x \ge 0$,

$$550x_1 + 600x_2 + 350x_3 + 400x_4 + 200x_5 \le 16G + \frac{179}{5}\,96.$$

The proposed plan attains this upper bound, so it is optimal. This weighted resource argument is an *optimality certificate*: the bound and its attainment establish the answer without trusting a solver.

G (h)	x_1	x_4	Optimal profit (PLN)
144	48/5	144/125	28704/5
192	48/5	384/125	32544/5

The strictly larger bound coefficients for products 2, 3, and 5 force their amounts to zero at an optimum; equality in the two resource bounds then fixes x_1 and x_4. Each continuous optimum is unique. Requiring whole products would change the model; rounding these quantities does not establish an integer optimum.

3 The general constrained problem

Let $x = (x_1, \ldots, x_n)^T \in \mathbb{R}^n$ be the decision vector. A constrained minimization problem is

$$\min_{x \in \Omega} f(x), \qquad \Omega = \{x \in \mathbb{R}^n : g_i(x) \leq 0 \ (i \in I), \ h_j(x) = 0 \ (j \in E)\}. \tag{2}$$

Here f is the objective, g_i are inequality-constraint functions, and h_j are equality-constraint functions; I and E are finite index sets. Bounds and sign restrictions must also be included in Ω when required.

Definition 3.1 (Feasibility and global optimality)**.** A point x is *feasible* if $x \in \Omega$. A feasible point x^* is a *global minimizer* if $f(x^*) \leq f(x)$ for every $x \in \Omega$. The value $f(x^*)$ is the optimal objective value.

An empty feasible set means that the requirements cannot be met simultaneously. Feasibility alone does not imply optimality. The optimal decision x^* and its objective value $f(x^*)$ are different objects, and multiple decisions may have the same value.

Remark 3.2 (Minimum versus infimum)**.** A finite lower bound need not be attained in a general optimization problem. For example, $\inf_{x>0} x = 0$, but no feasible point has value zero. A solver's output therefore requires both feasibility and an appropriate optimality or attainment argument.

4 From a real question to a mathematical model

1. **Choose variables and construct the model.** What can actually be controlled? What is given? State each variable's meaning, domain, and unit.

2. **Specify constraints.** Translate capacities, balances, required levels, and physical limits into equations or inequalities. Check their direction.

3. **Specify the objective.** State what "best" means, whether the criterion is minimized or maximized, and which units measure it.

4. **Select an algorithm.** Match the method to the model's structure: linearity, smoothness, convexity, dimensions, and continuous or discrete variables.

5. **Find and verify a solution.** Check feasibility and theoretical optimality conditions, then interpret the answer in the original setting.

6. **Perform post-optimal analysis.** Investigate how the answer depends on data and assumptions.

A correct solution to an inaccurate model can still be a poor real-world decision.

4.1 Units and assumptions

In a resource constraint, a coefficient measured in kg/item times a variable measured in items produces kg; its right-hand side must also be kg. Price in PLN/item times quantity in items produces revenue in PLN.

State whether quantities may be fractional. A count of indivisible objects requires integrality; permitting real values gives a continuous relaxation. State assumptions such as fixed coefficients, additive contributions, and unchanging prices. These are claims about the setting, not facts guaranteed by an optimization algorithm.

5 How optimization problems are classified

Constraints
Unconstrained or constrained optimization.

Variables over time
Static decisions or decisions evolving over time. Dynamic problems include control and functional optimization; this is broader than the algorithmic technique called dynamic programming.

Number of objectives
Single-objective or multiobjective optimization. Several competing criteria require an explicit way to compare trade-offs.

Function structure
Linear, nonlinear, quadratic, or convex optimization. Smoothness is another distinction; a nonlinear function may be smooth or nonsmooth.

Allowed values
Continuous, discrete, integer, or binary variables.

A linear program has a linear objective, linear constraints, and continuous variables. Adding integrality produces an integer linear program. Quadratic programming does not automatically mean convexity. For $f(x) = \frac{1}{2}x^T Q x + c^T x$ with symmetric Q, convexity requires Q to be positive semidefinite; a convex optimization problem also needs a convex feasible set.

6 Optimization roadmap

1. **Linear programming**: modeling examples, standard and canonical forms, graphical solutions, bases, simplex methods, duality, post-optimal analysis, and interior-point methods.
2. **Unconstrained nonlinear optimization**: local optimality conditions, iterative methods, convergence, line searches, steepest descent, Newton and quasi-Newton methods, conjugate gradients, and direct search.
3. **Constrained nonlinear optimization**: Lagrange multipliers, Karush–Kuhn–Tucker conditions, quadratic and convex problems, penalty and barrier methods, and interior-point methods.

The simplex method for linear programming and the Nelder–Mead simplex method are different algorithms. Their shared name does not make them interchangeable.

7 MATLAB refresher scope

Core MATLAB skills include vectors and matrices, programming constructs, built-in functions (including numerical methods), scripts, function files, parameter passing (including passing functions), and basic plots. Optimization Toolbox provides solvers for comparison with implemented algorithms.

8 Questions for revision

1. Which quantities in a model are decisions and which are parameters?
2. What distinguishes a feasible point, an optimal point, and an optimal value?
3. How would changing an inequality's direction change the model?
4. Why can fractional counts be mathematically feasible but physically unusable?
5. What must be checked after an algorithm returns a candidate solution?

9 Sources and editorial remarks

Primary source: `s4_w1.pdf`, PDF pages 1–6.
Source context: Folder labelled 2020; the slides contain a 2019/2020 schedule. Exact lecture date is not established.
Current course alignment: The supplied 2026–27 lecture `ProgMat_w1.pdf`, pp. 2–8, supports the modeling example in Section 2; the remaining topics are mapped in the current-course map. The historical roadmap and MATLAB scope are not confirmed current syllabus or assessment instructions.
Editorial scope: English synthesis of the mathematical content. Definitions and explanations expand the brief slides; historical schedules, assessment rules, and contact details are not current course instructions.

Current production data and verification. `ProgMat_w1.pdf`, pp. 2–7, supplies the five-product data. Page 4 states four grinding machines, while pp. 6–7 use three and 144 hours. Section 2 retains capacity as G and explicitly solves both readings. The weighted bound and uniqueness argument are added explanations. Page 8 indexes consumption by product first and resource second. The resource-row matrix convention in Section 2 is its transpose; changing index names consistently preserves the resource sums. The actual companion is modeling-patterns MATLAB script at `matlab/exercises/modeling_patterns_checks.m`; it checks the same rational data with absolute tolerance 10^{-10}. The new results were checked independently in exact rational arithmetic. MATLAB execution of this new script is unverified; execution evidence is maintained in `docs/matlab-verification.md`.

Polish equivalents are maintained in `docs/glossary.md`, including *zmienna decyzyjna*, *funkcja celu*, *ograniczenie*, *rozwiązanie dopuszczalne*, and *rozwiązanie optymalne*.

- `s4_w1.pdf`, pp. 1–2: historical administration and bibliography; not reproduced as current instructions.
- pp. 3–4: Sections 1, 3, 4, and 5, and the start of Section 6.
- pp. 5–6: remaining roadmap in Section 6 and MATLAB scope in Section 7.

Page numbers refer to PDF pages, each containing up to four slides.